Ders 4 Genişletilmiş Notu: Bilgi Teorisi ve Entropi, Veri Modelleri (Olasılık ve Markov) ve Move-to-Front Kodlama

Ders 4 Genişletilmiş Notu: Bilgi Teorisi ve Entropi, Veri Modelleri (Olasılık ve Markov) ve Move-to-Front Kodlama

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Bilgi Teorisine Giriş: Olasılık ve Bilgi
- 2. Bir Olayın Bilgisi (Self-Information)
- 3. Para Atma Örnekleri
- 4. Entropi
- 5. Entropi Örneği, Ortalama Mesaj Uzunluğu ve Artıklık
- 6. 2 Tabanında Logaritma
- 7. Fark Alarak Entropiyi Düşürmek: Rezidü Dizisi
- 8. Statik ve Dinamik Model
- 9. Sembolleri Bloklayarak Entropiyi Düşürmek
- 10. Veri Modelleri: Fiziksel Model ve Olasılık Modeli
- 11. Markov Model
- 12. Siyah-Beyaz Görüntü İçin Markov Model Örneği
- 13. İstatistiksel Yöntemlere Giriş ve Ön İşlemler
- 14. Move-to-Front (MTF) Kodlama
- 15. Kelimelerle Move-to-Front
- Örnek Sorular

1. Bilgi Teorisine Giriş: Olasılık ve Bilgi

Hoca bu derste entropi konusuna giriş yaptı. Bilgi teorisinin (information theory) tek başına bir dönemlik ders olabilecek kadar geniş ve teorik bir konu olduğunu, sıkıştırmada bizi en çok ilgilendiren kısmının entropi olduğunu söyledi. Konuyu gerçek hayattan örneklerle anlatmaya çalıştı.

Temel fikir şudur (ekrandan, 00:30:00): Tabiata baktığımızda bir olayın gerçekleşme ihtimali ne kadar düşükse o olayın taşıdığı bilgi o kadar fazladır. Tersi de doğrudur: Bir olayın gerçekleşme ihtimali ne kadar yüksekse o olay o kadar az bilgi içerir.

Hocanın örneği kardı. Hava sıfır derece ve kar yağıyor. Sıfır derecede kar yağma ihtimali çok yüksektir; bu yüzden “kar yağıyor” haberi önemli bir bilgi taşımaz. Tersini düşünelim: Hava 25-28 derece ve kar yağacağı söyleniyor. Buna inanmayız, çünkü ihtimali çok düşüktür. Ama gerçekten yağarsa olağanüstü bir tabiat olayı yaşanıyor demektir ve bu olayın taşıdığı bilgi çok yüksektir. (Slaytta bu örnek “20 derecede kar yağarsa” diye yazılmıştır.)

Bunun sıkıştırmadaki karşılığı şudur: Elimizde bir doküman olsun. Dokümanda bir sembol ne kadar fazla geçiyorsa olasılığı o kadar yüksektir. Olasılığı yüksek olanın taşıdığı bilgi az olduğu için onu daha az bitle ifade ederiz. Nadir geçen sembol ise daha fazla bitle ifade edilir. Hoca slayttaki cümleyi okuyup “bu dersin bütün amacı budur” dedi:

- Olasılık yüksek → bilgi düşük → daha az bitle kodla.
- Olasılık düşük → bilgi yüksek → daha fazla bitle kodla.

1.1 Sezgisel bir örnek: 100 tane a, 2 tane j

Bir dokümanda a harfinden 100 tane, j harfinden 2 tane olsun; toplam 102 karakter. a'nın olasılığı çok yüksek olduğu için onu belki 1 bitle, j'nin olasılığı çok düşük olduğu için onu 10 bitle kodlarız.

- a'lar: $100 \cdot 1 = 100$ bit.
- j'ler: $2 \cdot 10 = 20$ bit.
- Toplam: 120 bit.

Her karakteri ASCII ile 8 bitte bıraksaydık $102 \cdot 8 = 816$ bit gerekirdi. Böylece 816 bitlik veri 120 bite inmiş olur. Bu indirmede bize yardım eden şey bilgi teorisinde ortaya atılan fikirdir; biz bunu entropiyle ilişkilendireceğiz.

2. Bir Olayın Bilgisi (Self-Information)

Karşımıza a'nın çıkması bir olaydır, j'nin çıkması başka bir olaydır. Bir olayın olasılığını nasıl ölçeceğimizi biliyoruz: Bir bütün içinde o olaydan kaç adet varsa toplam sayıya oranlarız. Ama bilgi doğrudan ölçülebilen, sayılabilen bir şey değildir. Onu sayısal hâle getirmek için şu ölçü kullanılır (ekrandan, 00:30:00):

$$I(A) = \log_x \frac{1}{P(A)} = -\log_x P(A)$$

Burada $P(A)$ A olayının gerçekleşme ihtimali, $I(A)$ ise A olayının taşıdığı bilgidir. Formülün her parçasının bir gerekçesi vardır:

- **Neden $1/P(A)$?** Bilgi ile olasılık ters orantılıdır. Olasılık yüksekse bilgi düşük, olasılık düşükse bilgi yüksektir.
- **Neden logaritma?** Logaritma eldeki değeri küçültüp daha ölçülebilir hâle getirmek için kullanılır.
- **Neden 2 tabanı?** (ekrandan, 00:40:00) Bizim dünyamız dijital bir dünyadır; 1 ve 0'lardan oluşur. Entropinin birimi bit olacaktır ve bitin iki durumu vardır. Bu yüzden logaritma 2 tabanında alınır: $I(A) = \log_2 \frac{1}{P(A)}$. Slayta göre burada $I(A)$, kodlama sonucunda oluşan bit uzunluğu olarak ifade edilir.
- **Neden eksi?** Kesirle çalışmak zor olduğu için $P(A)$ paydan yukarı çıkarılır: $\log \frac{1}{P(A)} = \log P(A)^{-1} = -\log P(A)$. Olasılık 1'den küçük olduğu için $\log_2 P(A)$ negatiftir; baştaki eksi bilgiyi pozitif yapar. Bilgi negatif olamaz.

Örneğin 102 karakterin 100'ü a ise $P(a) = 100/102$'dir ve a'nın taşıdığı bilgi $-\log_2(100/102)$ olur.

3. Para Atma Örnekleri

3.1 Hilesiz para

Elimizde hatasız, hilesiz bir para var. Para atıldığında yazı ve tura gelme olasılıkları $1/2$'dir. Bu olayı kodlamak için kaç bit gerekir?

Hoca önce konuyu hiç bilmeyen birinin cevabını verdi: İki ihtimal var, ya yazı ya tura gelecek. Bir bit yeter; 1 tura, 0 yazı olsun. Sonra aynı sonucu formülle gösterdi (ekrandan, 00:40:00):

$$I(T) = \log_2 \frac{1}{1/2} = \log_2 2 = 1$$

Logaritmanın tabanı ile içindeki sayı aynı olduğunda sonuç 1'dir. Yazı için de aynı şekilde $I(Y) = 1$ çıkar. Yani bu durumu kodlamak için ortalama 1 bit gerekir.

3.2 Sahte para

Bu kez paranın sahte olduğu söyleniyor: $P(Y) = 1/8$, $P(T) = 7/8$. Yazıyı ve turayı kodlamak için kaç bit gerekir? Bulmamız gereken şey bilgidir (information).

- Yazı: $I(Y) = -\log_2 \frac{1}{8} = \log_2 8 = \log_2 2^3 = 3$ bit.
- Tura: $I(T) = \log_2 \frac{8}{7} \approx 0{,}193$ bit (ekrandan, 00:40:00; python ile doğrulandı).

Hoca burada akla gelecek soruyu kendisi sordu: "Yazı için 3 biti anladık da, turayı kodlamak için 0,193 bit ne demek?" Tabii ki böyle bir bit yoktur; bir bit ya vardır ya yoktur. Bu teoride yapılan hesaptır. Pratikte yazı için 3 bit kullanılırsa tura için de 1 bit kullanılır, çünkü 0,193 bitlik bir yer yoktur.

> **Hocanın vurgusu:** Sıkıştırma performansının hiçbir zaman %100 olmamasının sebeplerinden biri budur. Teorideki hesaplara göre ideal sonuç çıkabilir, ama pratikte kesirli bit olmadığı için bunu aynen uygulayamayız.

4. Entropi

4.1 Tek olaydan çok olaya

Şimdiye kadar tek bir olaydan, parayı bir kez atmaktan söz ettik. İstatistik derslerinde "parayı arka arkaya on kez atınca" diye sorulur; bu on farklı olaydır. Bir metinde de durum böyledir: Metinde A, B, C, D, E harfleri vardır, ya da resimde 28. renk kodu, 55. renk kodu vardır. Bir dokümanda A'nın karşımıza çıkması bir olay, B'nin çıkması başka bir olaydır.

Birbirinden bağımsız n olaydan bir set oluşturup bu seti S ile gösterirsek (ekrandan, 00:40:00): $S = \bigcup_{i=1}^{n} A_i$.

4.2 Entropi formülü

S örnek seti içindeki ortalama bilgiyi (self-information) elde etmek istersek şu formülü kullanırız: Her bir olayın gerçekleşme ihtimalinin o olayın taşıdığı bilgiyle çarpımlarının toplamı, bu olay setinin ortalama bilgisini, yani entropisini verir.

$$H = \sum_{i=1}^{n} P(A_i)\, I(A_i)$$

$I(A_i)$ yerine $-\log_2 P(A_i)$ yazılırsa:

$$H = -\sum_{i=1}^{n} P(A_i) \log_2 P(A_i)$$

Hoca bazı kaynakların entropiyi E ile gösterdiğini, ama genelde H kullanıldığını söyledi. Kısacası entropi, her olayın olasılığı ile o olayın taşıdığı bilginin çarpımlarının toplamıdır.

> **Ders sonrası not:** Hoca 00:37:43'te slayttaki formülün başına eksi koymayı unuttuğunu söyleyip eksi ekledi, sonra 00:43:17'de "burada yanlış söyledim" diyerek eksinin $I(A_i)$ ile yazılan formülde değil, $\log_2 P(A_i)$ ile yazılan formülde olması gerektiğini açıkladı. Doğrusu yukarıdaki iki eşitliktir: I zaten pozitiftir, eksi yalnız $\log_2 P$ ile yazılan biçimde gerekir.

4.3 Entropi ne söyler?

Entropi nedir diye sorulduğunda sözel cevap şudur: **ortalama binary sembol uzunluğu.** Elimizdeki set bir görüntü, bir metin ya da başka bir veri olabilir. Onu sıkıştırıp kodladığımızda her bir sembolü ortalama kaç bitle kodlayacağımızı entropi değeri söyler. Slaytta: "Bu olayları kodlamaya geçtiğimizde entropy bize olması gereken ideal ortalama binary sembol uzunluğunu verecektir."

Örneğin bir sayfayı sıkıştıracağız. Sayfadaki her karakterin olasılığını ve taşıdığı bilgiyi çıkarıp entropiye koyarız. Çıkan sayı, bir sembolü kodlamak için ortalama kaç bite ihtiyaç olduğunu söyler. $H = 4{,}5$ çıksın: Bazı semboller 1 bitle, bazıları 2, 3, 6, 7 bitle kodlanır, ama hepsinin ortalaması 4,5 bit olur.

> **Hocanın vurgusu:** "Bu entropiyi kesinlikle unutmayın."

4.4 Shannon ve ideal sıkıştırma

Veri sıkıştırmada ilk çalışmaları yapanlardan biri olan Shannon'a göre (ekrandan, 00:44:10) en iyi kayıpsız sıkıştırmada kodlanan sembollerin ortalama uzunluğu, teoride bulduğumuz kaynağın entropisine eşit olmalıdır. Ancak bu ideal durum pratikte mümkün olmamaktadır. Hoca örnek verdi: Entropi 1,64 çıksa, ne yaparsak yapalım bir sembolü 1,64 bitle saklayamayız; 1 ya da 2 bitle saklayabiliriz. Entropideki değer teoride geçerlidir, gerçek dünyada aynen saklanamaz.

> **Ek bilgi (derste anlatılmadı):** Bu sonuç bilgi teorisinde Shannon'ın kaynak kodlama teoremi olarak bilinir. Kayıpsız ve her sembolü ayrı kodlayan bir kodun ortalama uzunluğu L için $H \leq L$ sınırı vardır; uygun bir kodla $L < H + 1$ sağlanabilir. Sembolleri bloklar hâlinde kodlamak, sembol başına ortalama uzunluğu entropiye daha da yaklaştırır.

5. Entropi Örneği, Ortalama Mesaj Uzunluğu ve Artıklık

5.1 Dört sembollü kaynak

Kaynak alfabesi a_1, a_2, a_3, a_4'ten oluşsun ve olasılıklar $P(a_1) = 0{,}49$, $P(a_2) = 0{,}25$, $P(a_3) = 0{,}25$, $P(a_4) = 0{,}01$ olsun (ekrandan, 00:44:10). Bu olasılıklar her sembolün adedinin toplam sayıya bölünmesiyle bulunur. Soru: Bu dokümanı ortalama kaç bitle kodlarız?

Entropi formülünde i birden 4'e gider, çünkü dört olay vardır:

$$H = -(0{,}49 \log_2 0{,}49 + 0{,}25 \log_2 0{,}25 + 0{,}25 \log_2 0{,}25 + 0{,}01 \log_2 0{,}01) = 1{,}57 \text{ bit}$$

Ara değerler (ara adım eklendi; python ile doğrulandı): $0{,}49 \cdot 1{,}029 \approx 0{,}504$, iki tane $0{,}25 \cdot 2 = 0{,}5$, $0{,}01 \cdot 6{,}644 \approx 0{,}066$; toplam $\approx 1{,}571$. Bu set ortalama 1,57 bitle kodlanabilir.

> **Ders sonrası not:** Hoca 00:42:29'da dördüncü olasılığı "0.1" diye okudu; slayttaki ve hesaptaki değer 0,01'dir (dört olasılığın toplamı 1 olmalıdır).

5.2 Ortalama mesaj uzunluğu

1,57 tam sayı olmadığı için pratikte farklı kodlanır. Hoca dört sembolün her birine 2 bitlik bir kod verdi: `00`, `01`, `10`, `11`. Her sembolün bit sayısı 2 olduğu için ortalama mesaj uzunluğu $S = 2$ bittir.

5.3 Artıklık (redundancy)

İlk haftada görülen terimlerden biri olan artıklık (redundancy) burada tanımlanır. Teori "1,57 bitle kodlayacaksın" der; pratikte her sembol 2 bit çıkmıştır. İkisi arasındaki farkın mutlak değeri artıklıktır:

$$R = |S - H| = |2 - 1{,}57| = 0{,}43$$

Artıklığın yorumu:

- **Sıfıra yakınsa:** En ideal sıkıştırma kullanılmıştır. Verinin içinde kodlanabilecek, sıkıştırılabilecek başka bir şey kalmamıştır. Üstüne ikinci bir sıkıştırma yöntemi uygulamaya gerek yoktur.

- **Büyükse (örneğin 4):** Veriye uygun bir yöntem seçilmemiştir. Ya yöntem değiştirilir ya da arkasından ikinci bir sıkıştırma yöntemi uygulanarak artıklık küçültülür. Ne kadar küçültülürse o kadar iyi sıkıştırma yapılmış olur.

Hoca bu slayttan çıkan üç kavramı şöyle topladı: **entropi** teoride veriyi ortalama kaç bitle kodlayacağımızı söyler; S pratikte gerçekten kaç bitle kodladığımızdır; ikisi arasındaki mutlak fark **artıklıktır.** "Bütün her şey bunun üzerine kurulu."

6. 2 Tabanında Logaritma

Hesap makinelerinde 2 tabanında logaritma yoktur, 10 tabanında (ya da doğal) logaritma vardır. Dönüşüm şöyle türetilir (ekrandan, 00:50:00):

$$\log_2 x = q \;\Rightarrow\; x = 2^q \;\Rightarrow\; \ln x = \ln 2^q = q \ln 2 \;\Rightarrow\; q = \frac{\ln x}{\ln 2}$$

Aynı şey 10 tabanı için de geçerlidir:

$$\log_2 x = \frac{\log_{10} x}{\log_{10} 2}$$

Yani hesap makinesinde bir sayının 2 tabanında logaritması için $\log_{10} x$, $\log_{10} 2$'ye bölünür.

7. Fark Alarak Entropiyi Düşürmek: Rezidü Dizisi

7.1 Orijinal dizinin entropisi

Slayttaki örnek (ekrandan, 00:50:00):

```
1 2 3 2 3 4 5 4 5 6 7 8 9 8 9 10
```

Bu sayı dizisi kodlanacak olursa ortalama bit uzunluğu ne olur? Bu soru, teoride kaç bit gerektiğini, yani entropiyi sorar. Önce olayların ne olduğuna karar verilir: Her farklı sembolün karşımıza çıkması bir olaydır. 1'den 10'a kadar 10 farklı sembol, yani 10 olay vardır. Toplam 16 sembol vardır.

- Bir kez geçenler: 1, 6, 7, 10 → her biri $1/16$.
- İki kez geçenler: 2, 3, 4, 5, 8, 9 → her biri $2/16 = 1/8$.

$$H = -\left[4 \cdot \tfrac{1}{16} \log_2 \tfrac{1}{16} + 6 \cdot \tfrac{1}{8} \log_2 \tfrac{1}{8}\right] = 4 \cdot \tfrac{1}{16} \cdot 4 + 6 \cdot \tfrac{1}{8} \cdot 3 = 1 + 2{,}25 = 3{,}25 \text{ bit}$$

(ekrandan, 00:54:35; python ile doğrulandı.) Ortalama 3,25 bit gerekir. Kodlandığında bazı semboller 1, bazıları 3 ya da 4 bitle kodlanacak, ortalaması 3,25'e gelecektir. Gerçek dünyada S belki 4 çıkacak, aradaki fark da artıklık olacaktır.

> **Ders sonrası not:** Slaytta $P(1) = P(6) = P(7) = P(10) = 1/6$ yazıyor (00:54:35); doğrusu $1/16$'dır. Hoca anlatırken $1/16$ dedi. Aynı karedeki $H = \sum P_i \log_2 P_i$ yazımında da baştaki eksi eksiktir.

7.2 Fark almak

Hoca "Bu diziyi başka nasıl işleyerek entropiyi düşürebilirim?" diye sordu. Cevap fark almaktır: İlk eleman olduğu gibi yazılır, sonraki her eleman kendinden önceki elemandan çıkarılır. $2 - 1 = 1$, $3 - 2 = 1$, $2 - 3 = -1$, ... Böylece "residual sequence" (rezidü dizisi) elde edilir (ekrandan, 01:00:00):

```
1 1 1 -1 1 1 1 -1 1 1 1 1 1 -1 1 1
```

Bu dizinin entropisi elbette düşer: Önceki dizide 10 farklı sembol vardı, burada yalnız iki sembol (1 ve -1), yani iki olay vardır. 13 tane 1, 3 tane -1 vardır:

$$P(1) = \tfrac{13}{16}, \quad P(-1) = \tfrac{3}{16}, \quad H = -\left(\tfrac{13}{16}\log_2\tfrac{13}{16} + \tfrac{3}{16}\log_2\tfrac{3}{16}\right) \approx 0{,}70 \text{ bit}$$

(python ile 0,696 bulundu.) 3,25 bitlik dizi, önünde yapılan bir işlemle teoride 0,70 bitle kodlanır; pratikte bu 1 bite karşılık gelir.

7.3 Alıcı orijinal veriye dönebilir mi?

Slayttaki soru buydu: "Alıcı bu diziden orijinal veriye dönüş yapabilir mi?" Alıcı sıkıştırılmış veriyi açtığında rezidü dizisini elde eder ve onu orijinal veri sanabilir. Hâlbuki orijinal veri bu değildir. Bu yüzden karşı tarafa iki şey söylenmelidir: verinin orijinal değil, orijinalin rezidüsü alınarak elde edilmiş bir dizi olduğu ve nasıl geri açılacağı. Karşı tarafa söylenmesi gereken her şey sıkıştırılmış verinin üstündeki başlık (header) bilgisine yazılır. Ya da iki taraf önceden anlaşmıştır; ikisi de kuralı bilir.

Slayta göre sekansın modelinin çıkarılması gerekir (ekrandan, 01:04:45):

- x_n: orijinal dizinin n. elemanı,
- r_n: rezidü dizisinin n. elemanı,
- $x_n = x_{n-1} + r_n$.

Hoca açmayı adım adım yaptı. İlk eleman zaten orijinal değer olarak konmuştur: $x_1 = 1$.

- $x_2 = x_1 + r_2 = 1 + 1 = 2$
- $x_3 = x_2 + r_3 = 2 + 1 = 3$
- $x_4 = x_3 + r_4 = 3 + (-1) = 2$

Böylece orijinal dizi geri elde edilir.

7.4 Neden böyle bir şey yapılır?

Orijinal diziyi doğrudan kodlasaydık entropi 3,25 olacaktı. Bir kodlama yapıp ortalama mesaj uzunluğunu bulduğumuzda entropiyle S arasındaki farkın sıfıra yakın olması gerekirken oldukça büyük olduğunu görecektik. "Demek ki bu diziyi daha iyi sıkıştırabilirim" diyecektik. İşte o zaman arka planda nasıl bir algoritma kurmamız gerektiğini düşünürüz: Dizinin gidişine bakınca ardışık elemanlar arasındaki farkların $+1$, -1 ya da $+2$, -2 gibi küçük değerler olduğunu görürsek rezidüyü alır ve onu sıkıştırırız.

Rezidü dizisi 0,70 bit entropiye sahiptir ve iki farklı durum olduğu için pratikte 1 bitle kodlanır. Hoca $1 - 0{,}70 = 0{,}3$ farkının sıfıra yakın olduğunu, artık artıklık kalmadığını ve daha ideal bir sıkıştırma bulunduğunu söyledi. Tek sorun, açan kişinin bunun orijinal dizi olmadığını bilmesi gerektiğidir. Veri iletişimindeki iki uçta da bu formül vardır; alan taraf formülü uygulayıp orijinali elde eder.

8. Statik ve Dinamik Model

Slayta göre (ekrandan, 01:04:45) bu model **statik model** olarak adlandırılır, çünkü parametreler n değiştikçe değişmez. Verinin karakteristiğine bağlı olarak parametreler n ile değişiyorsa **dinamik model** adını alır.

Hoca bunu açtı: $x_n = x_{n-1} + r_n$ modelinde n 1, 2, 3 diye ilerler ama yapılan işlem hep aynıdır; her adımda yalnız x ile r toplanır, başka bir işlem yapılmaz. Dinamik modelde ise n'nin değerine göre yapılan işlem değişebilir. Örneğin "n çift değerlerdeyken şunu, tek değerlerdeyken bunu yap" ya da "n şu aralıktaysa şunu, bu aralıktaysa bunu yap" diyen bir model dinamiktir.

9. Sembolleri Bloklayarak Entropiyi Düşürmek

9.1 Problem

Hoca ara vermeden önce konunun bütününü bitirmek istedi ve öğrencilerden yardım istedi. Slayttaki dizi (ekrandan, 01:10:00):

```
1 2 1 2 3 3 3 3 1 2 3 3 3 3 1 2 3 3 1 2
```

Bu dizi ortalama kaç bitle kodlanabilir? İlk akla gelen cevap: Üç sembol var (1, 2, 3), her birinin olasılığını bulur, entropiyi hesaplarız. Bu doğrudur, ama artıklığın yüksek çıktığı görülecektir. Hoca "önceki örnekte rezidü alarak işin rengini değiştirmiştik; burada da öyle bir şey yapılabilir mi?" diye sordu.

9.2 Öğrenci önerisi: RLE

Soru: Bir öğrenci tekrar sayısı üzerinden gidilebileceğini önerdi: örneğin 3 arka arkaya dört kez geldiği için dört tane 3 yazmak yerine başına 4 yazılabilir. **Cevap:** Hoca bunun RLE olduğunu söyledi ve diziyi RLE ile kodladı (ekrandan, 01:10:00):

```
1 1  1 2  1 1  1 2  4 3  1 1  1 2  4 3  1 1  1 2  2 3  1 1  1 2
```

Orijinal 20 bayttır, RLE çıktısı 26 bayttır (13 çift). Bu veriyi RLE ile sıkıştırmak sıkıştırmak değil, genişletmektir. RLE'nin en başarılı olduğu durum sembollerin arka arkaya çok tekrar etmesidir; burada çok farklı semboller arka arkaya geldiği için RLE otomatik olarak elenir.

Hoca geçen haftaki bayrak baytlı RLE versiyonunu da denedi: Her 8 değer için önce bir kontrol baytı yazılır; bitteki 0 arkasından gelen değerin gerçek değer, 1 ise run olduğunu söyler. Örneğin 1 ve 2 için bayrak bitleri 0, `4 3` için önce 1 (run), sonra 0 (değer) olur. Bu versiyon tek tek gelen sayılar için run yazıp veriyi genişletmeyi önler, ama bu dizide yine uzatır. Hoca başka bir şey görmemizi istediğini söyledi.

9.3 Tekrar eden desenler

Diziye bakınca belirli desenler görülür: `12` tekrar ediyor, `33` tekrar ediyor. İkişerli alındığında yalnız bu iki desen çıkar. Üçerli alınırsa (`121`, `233`, `331`, …) çok daha değişik sayılar çıkar. Dörderli alınırsa `1212`, `3333`, `1233`, `3312` gibi dört farklı sembol çıkar. Olay sayısı ne kadar az olursa o kadar az bitle kodlanır.

9.4 Tek tek sembollerle entropi

Dizide 20 sembol vardır. 1 ve 2 beşer kez, 3 on kez geçer (python ile sayıldı):

$$P(1) = P(2) = \tfrac{5}{20} = \tfrac{1}{4}, \quad P(3) = \tfrac{10}{20} = \tfrac{1}{2}$$

$$H = -\left(2 \cdot \tfrac{1}{4} \log_2 \tfrac{1}{4} + \tfrac{1}{2} \log_2 \tfrac{1}{2}\right) = 1 + 0{,}5 = 1{,}5 \text{ bit/sembol}$$

Slayta göre sekans 20 sembolden oluştuğu için kodlamak için gerekli bit sayısı $20 \cdot 1{,}5 = 30$'dur (ekrandan, 01:15:40). Üç sembol sabit uzunluklu kodla 2 bitle kodlanırsa (`00`, `01`, `10` gibi) ortalama mesaj uzunluğu $S = 2$, artıklık $|2 - 1{,}5| = 0{,}5$ olur.

> **Ders sonrası not:** Hoca 01:13:01'de "1'den 4 tane var, 2'den de 4 tane var" dedi; dizide 1 ve 2 beşer kez geçer. Olasılık $1/4$ sonucu doğrudur. Ayrıca 01:14:15'te altyazıda hem 2 hem 3 için `01` kodu geçiyor; üç sembol farklı kodlar almalıdır.

9.5 İkişerli bloklarla entropi

Acaba 0,5'lik artıklık daha da indirilebilir mi? Öğrencinin işaret ettiği gibi `12` ve `33` blokları alınır. Dizide 5 tane `12` ve 5 tane `33`, toplam 10 blok vardır.

$$P(12) = P(33) = \tfrac{5}{10} = \tfrac{1}{2}, \quad H = -2 \cdot \tfrac{1}{2} \log_2 \tfrac{1}{2} = 1 \text{ bit/blok}$$

Slayta göre sekans 10 sembolden (bloktan) oluştuğu için kodlamak için gerekli bit sayısı 10'dur (ekrandan, 01:15:40). Normal kodla `12`'lere 0, `33`'lere 1 verilirse 1 bit harcanır, $S = 1$ olur. Entropi 1, S 1 olduğu için artıklık sıfırdır. Böylece en ideal sıkıştırma bulunmuş olur. 30 bit 10 bite inmiştir.

Ders sonrası not: Slaytta $P(12) = P(33) = 1/4$ yazıyor ve hoca önce $5/20$ dedi. 20 sayı 10 bloğa ayrıldığı için olasılık $5/10 = 1/2$'dir; karede hocanın $5/20$'nin yanına $5/10$ yazıp işaretlediği görülüyor (01:15:40). $H = 1$ bit/blok sonucu $1/2$ ile doğrudur (python ile doğrulandı).

9.6 Ara öncesi özet

Hoca bu bölümü şöyle bağladı: Entropi bir mesajı ya da veriyi kodlamak için gereken ortalama bit uzunluğudur, ama teoriktir. Bir sıkıştırma yöntemiyle sembollere kod değerleri bulunca ortalama mesaj uzunluğu S hesaplanır. H ile S arasındaki fark artıklıktır ve sıfıra yakın olmalıdır. Değilse ideal yöntem seçilmemiştir, içinde hâlâ sıkıştırılacak veri vardır: ya yöntem değiştirilir ya da arka arkaya iki ya da üç yöntem uygulanır. Sıkıştırma yerine genişleme oluyorsa, yani çıktı orijinalden büyükse, tamamen yanlış yoldayız demektir; veriye bakış açısı değiştirilip uygun bir yöntem seçilmelidir.

10. Veri Modelleri: Fiziksel Model ve Olasılık Modeli

10.1 Neden model?

Veriyi sıkıştırırken verinin içeriğini tanımamız gerekir. İçeriğe göre fiziksel model, olasılık modeli ya da Markov model uygulanabilir (ekrandan, 01:40:00). Slayta göre verinin modelini iyi ve doğru oluşturmak, kaynağın entropisini tahmin etmek ve etkili bir sıkıştırma algoritması seçmek için önemlidir. Model ne kadar iyi kurulursa entropi tahmini o kadar doğru ve gerçekçi olur.

10.2 Fiziksel model

Fiziksel modelde verinin nasıl üretildiği bilinir. Hocanın örnekleri:

- Bir ses kaydını ön işleme sokmak için kaydın hangi ortamda alındığını bilmek gerekir: sessiz bir odada mı, kalabalık bir ortamda mı? İki üç ses varsa içinden istenen kişinin sesi mi çekilecek?
- Bir hanenin elektrik tüketimini modellemek için hanede kaç kişi yaşadığı ve evde bulunma saatleri bilinirse daha iyi bir model kurulur.

Hoca bunların dersin içeriğiyle çok ilişkili olmadığını, derste en fazla olasılık modeli ve Markov modelin kullanılacağını söyledi.

10.3 Olasılık modeli

Olasılık modeli istatistiksel bir modeldir. Şimdiye kadar bahsedilen bütün modeller olasılık bilgisini kullanır; entropiden söz ediyorsak olasılık bilgisine ve olasılık modellerine mecburuz.

Bağımsız ve bağımlı olasılık. Bir dokümanda A harfinin gelme olasılığı sorulursa bu bağımsız bir olaydır; A'nın tek başına gelmesi sorulur. Ama "A'dan önce B gelmişse A'nın gelme olasılığı nedir?" diye sorulursa bağımlı bir olasılık vardır. Bağımlı olasılıklar Markov modelde değerlendirilir.

Slayta göre (ekrandan, 01:43:40) olasılık modelinin iki biçimi vardır:

- **Ignorance model:** En basit istatistiksel modeldir. Kaynak tarafından üretilen her sembolün birbirinden bağımsız ve eşit olasılıkla üretildiği durumdur. Kaynak hakkında hiçbir bilgi yoksa kullanılır. Hoca bunu şöyle açtı: Dokümanın içini göremiyoruz; A'dan, B'den, C'den kaç tane olduğunu bilmiyoruz. O zaman her sembolün olasılığı eşittir: İngilizce için $1/26$, Türkçe için $1/29$.
- **Probability model:** Eşit olasılık yerine her sembolün meydana gelme sıklığı göz önüne alınarak olasılık hesaplanır. Kaynağın içeriğini biliriz; her sembolden kaç adet olduğunu sayarız. Semboller yine birbirinden bağımsızdır (A'nın gelmesi B'ye bağlı değildir), ama olasılıkları farklıdır. Alfabe $\{a_1, a_2, \dots, a_m\}$, olasılıklar $P = \{P(a_1), P(a_2), \dots, P(a_m)\}$. İstatistiksel veri sıkıştırma yöntemleri bu modeli kullanır.

Probability modelde olasılıklar bulunduktan sonra entropi hesaplanır, veri bir sıkıştırma yöntemine sokulur, ortalama mesaj uzunluğu ve artıklık bulunur; sıkıştırmanın iyi olup olmadığına karar verilir.

> **Hocanın vurgusu:** Probability modelde kaynak hakkında bilgi vardır; içerik görülür, olasılıklar bağımsız ve farklıdır. Ignorance modelde kaynak hakkında bilgi yoktur; olasılıklar bağımsız ve birbirinin aynıdır.

11. Markov Model

11.1 Tanım

Slayta göre (ekrandan, 01:50:00) Markov modelde sembollerin olasılıkları birbirine bağımlı olarak hesaplanır. Model adını Rus matematikçi A. A. Markov'dan alır. Kayıpsız sıkıştırma yöntemleri için kullanılır ve "Discrete Time Markov Chain" olarak da bilinir.

Markov model kullanıldığında A'nın olasılığı, A'nın birlikte kullanıldığı sembollere bağlı olarak hesaplanır. Kaynakta A'dan önce boşluk varsa "boşluk ve A"nın birlikte olma olasılığı, A'dan önce K varsa "K ve A"nın birlikte olma olasılığı ayrı ayrı çıkarılır.

11.2 k'ıncı dereceden Markov model

$\{x_n\}$ gözlemlenen bir sekans olsun. Bu sekans k'ıncı dereceden (k-th order) Markov model olarak şöyle gösterilir:

$$P(x_n \mid x_{n-1}, \dots, x_{n-k}) \approx P(x_n \mid x_{n-1}, \dots, x_{n-k}, \dots)$$

Yani geçmişteki k sembolün bilgisi, işlemin geçmişteki tüm bilgisine yaklaşık olarak eşittir.

Hoca tahtaya birkaç kelime yazıp (ekrandan, 01:50:00) ikinci dereceden modelde A'nın olasılığını hesaplamak için A'dan önceki iki karakterin bütün durumlarının çıkarılması gerektiğini gösterdi: önünde "boşluk B" olan A, "A B" olan A, "boşluk K" olan A gibi. Bu durumların her birinin olasılığı ayrı ayrı hesaplanır.

11.3 Neden küçük k?

$k = 5$ alınsın: x_n'nin olasılığı x_{n-1}'den x_{n-5}'e kadar geçmişe göre hesaplanır. k sonsuz bırakılırsa, örneğin bir dokümanın son karakteri olan A'nın olasılığı geçmişteki bütün karakterlere göre hesaplanır. Böyle bir dizi karşımıza belki ancak sayfanın kopyası çıktığında, yani bir cümle, paragraf ya da sayfa olduğu gibi kopyalandığında tekrar gelir. Bu yüzden olasılığı çok çok küçüktür.

Hoca bunu bir harf dizisi üzerinden anlattı (`K L M N O A Z`): Z'nin olasılığı, önce yalnız A'ya, sonra A ve O'ya, sonra A, O, N'ye bağlı olasılıklarla genişletilir. Geçmiş uzadıkça bu olasılıklar o kadar küçülür ki eklenen terimlerin etkisi çok küçük kalır; sonuç kısa geçmişle hesaplanana yaklaşık olarak eşit olur. Hoca bunun ayrıntısının NLP derslerinde görüldüğünü söyledi. Bir karakterin olasılığını geçmişindeki bütün karakterlerle birlikte hesaplamaya ne işlem gücü ne makinenin gücü yeter. Bu yüzden çalışmalarda k 2 ya da 3 alınır; model de böylece daha basit olur.

12. Siyah-Beyaz Görüntü İçin Markov Model Örneği

12.1 Durumlar ve geçişler

Örneği basit tutmak için siyah-beyaz (binary) bir görüntü alındı (ekrandan, 02:00:00). Hoca tahtada bazı pikselleri siyah, bazılarını beyaz boyadı. Amaç bu görüntüyü Markov modele göre kodlamak ve olasılıkları belirlemektir; ayrıca Markov model ile olasılık modeli arasındaki fark gösterilecektir.

Bir pikselin beyaz mı siyah mı olacağı, ondan önceki pikselin beyaz mı siyah mı olduğuna bakılarak tahmin edilir. İki durum (state) vardır: S_w (white) ve S_b (black). Dört geçiş vardır:

- S_w'deyken beyaz gelirse S_w'de kalınır: $P(w|w)$.
- S_w'deyken siyah gelirse S_b'ye geçilir: $P(b|w)$.
- S_b'deyken siyah gelirse S_b'de kalınır: $P(b|b)$.
- S_b'deyken beyaz gelirse S_w'ye geçilir: $P(w|b)$.

Durumların olasılıkları $P(S_w)$ ve $P(S_b)$'dir.

Hoca derece arttıkça durumların nasıl çoğalacağını anlattı: Daha yüksek derecede "siyahtan önce beyaz beyaz", "siyahtan önce siyah siyah", "siyahtan önce beyaz siyah", "siyahtan önce siyah beyaz" gibi durumlar oluşur; üçüncü derecede her piksel için $2^3 = 8$ farklı durum ve geçiş olur. Resim grayscale, color table ya da true color olursa durum sayısı renk sayısıyla artar. Bu yüzden Markov modeller çok daha iyi bir sıkıştırma sunar, ama kodlaması daha zor, meşakkatli ve yapısı daha karmaşıktır.

> **Ders sonrası not:** Hoca bu iki durumlu modeli "ikinci dereceden Markov model" diye andı. Yalnız bir önceki pikseli kullanan model, standart terminolojide büyük olasılıkla birinci dereceden (first-order) Markov modeldir; k. derecede ikili görüntü için 2^k bağlam olur. Hoca ayrıca 5 bitlik grayscale için "5 state", color table için "255 state" dedi; 5 bitte $2^5 = 32$, 256 renkli color table'da 256 renk vardır.

12.2 Markov modelde entropi

Olasılık modelinde entropi $H = -\sum P_i \log_2 P_i$ idi. Markov modelde formül biraz değişir (ekrandan, 02:00:00):

$$H = \sum_{i=1}^{M} P(S_i)\, H(S_i)$$

Burada M durum sayısıdır. Her durumun olasılığı o durumun entropisiyle çarpılır. Bilginin yerine durumun entropisi gelir; durumun entropisi de olasılık modeliyle hesaplanır:

$$H(S_w) = -\left[P(w|w)\log_2 P(w|w) + P(b|w)\log_2 P(b|w)\right]$$

$$H(S_b) = -\left[P(b|b)\log_2 P(b|b) + P(w|b)\log_2 P(w|b)\right]$$

Verilen geçiş olasılığından diğeri bulunur, çünkü bir durumdan çıkan olasılıkların toplamı 1'dir: $P(w|w) = 1 - P(b|w)$, $P(b|b) = 1 - P(w|b)$.

12.3 Sayısal örnek

Slayttaki örnek (ekrandan, 02:10:00): $P(S_w) = 0{,}8$, $P(S_b) = 0{,}2$, $P(w|b) = 0{,}3$, $P(b|w) = 0{,}01$. Hem Markov model hem olasılık modeli ile sistemin entropisini hesaplayınız. (Hoca slayttaki "İstatistiksel Model" ifadesini "Olasılık Model" olarak düzeltti.)

$P(S_w) = 0{,}8$ şu demektir: Elimizde 10 piksel varsa 8'i beyazdır. $P(S_b) = 1 - 0{,}8 = 0{,}2$. $P(b|b) = 1 - 0{,}3 = 0{,}7$, $P(w|w) = 1 - 0{,}01 = 0{,}99$.

Olasılık modeli:

$$H = -(0{,}8\log_2 0{,}8 + 0{,}2\log_2 0{,}2) = 0{,}722 \text{ bit}$$

Normal hayatta siyaha 0, beyaza 1 verilirse ortalama mesaj uzunluğu $S = 1$ olur; artıklık $|1 - 0{,}722| = 0{,}278$'dir (ara adım eklendi; karede $H = |1 - 0{,}722|$ diye yazılmış).

Markov model:

$$H(S_b) = -(0{,}3\log_2 0{,}3 + 0{,}7\log_2 0{,}7) = 0{,}881 \text{ bit}$$

$$H(S_w) = -(0{,}01 \log_2 0{,}01 + 0{,}99 \log_2 0{,}99) = 0{,}081 \text{ bit}$$

$$H = P(S_b)\,H(S_b) + P(S_w)\,H(S_w) = 0{,}2 \cdot 0{,}881 + 0{,}8 \cdot 0{,}081 = 0{,}176 + 0{,}065 = 0{,}241 \text{ bit}$$

(python ile doğrulandı: 0,2409.)

12.4 Yorum

Markov model daha düşük entropi (0,241 bit, olasılık modelinde 0,722 bit) verdiği için siyah-beyaz görüntü Markov modelle sıkıştırılırsa daha iyi bir sıkıştırma olacaktır. Ama bu örnekte ikisi de pratikte 1 bitle kodlanacaktır; 0,7 ya da 0,24 bit diye bir şey olmadığı için hangisinin seçildiği fark etmez. Hoca bu örneği yalnız işlemi hızlı yapmak için iki durumla verdiğini söyledi.

Çok daha fazla renk olduğunda ve geçmişteki semboller de kullanıldığında Markov model olasılık modelinden hissedilir derecede daha küçük entropi verir; örneğin biri 0,7 çıkarken diğeri belki 3 çıkar ve fark açıkça görülür. Örneğin 256 renkli bir color table'da 256 durum olur; tek rengin değil, geçmişteki renklerle birlikte olasılığı kullanılırsa işlem yükü daha da artar.

> **Hocanın vurgusu:** Markov model her zaman olasılık modelinden daha iyi sonuç verir, ama daha karmaşıktır ve kodlaması daha zordur. Bazı verilerde olasılık modeli ile Markov model arasında abartılacak bir fark yoktur. Olasılık modelinde iki durum (siyah, beyaz), Markov modelde dört durum (beyazdan önce beyaz, beyazdan önce siyah, siyahtan önce siyah, siyahtan önce beyaz) vardır.

12.5 Bilgi teorisinin özeti

Hoca bilgi teorisini bitirirken şunu söyledi: Bu dersin sonunda entropi, ortalama mesaj uzunluğu ve artıklık sorulsa bilinmelidir. Her şey bunların üzerine kurulmuştur; geri kalan yöntemler teferruattır. A kişisi bir yöntem, B kişisi başka bir yöntem çıkarmıştır. Amaç her zaman artıklığı en aza indirmektir. Bunun için en uygun model ve algoritma seçilmelidir; en iyi algoritmayı seçmek için de verinin içi tanınmalıdır.

13. İstatistiksel Yöntemlere Giriş ve Ön İşlemler

Ardından istatistiksel yöntemlere geçildi. Hoca bu konunun bugün bitmeyeceğini, yalnız giriş yapılacağını söyledi. Günümüzde kullanılan her yöntemin içinde karşımıza Huffman, aritmetik kodlama ya da PPM gibi yöntemler çıkar.

> **Ders sonrası not:** Hoca 02:12:33'te "fakslarda MNP5 ve 7 algoritmaları var" dedi. MNP5 ve MNP7 büyük olasılıkla modem bağlantılarında kullanılan Microcom Network Protocol sürümleridir; önceki derste MNP5 de modem protokolü olarak anılmıştı.

Bu bölümde gösterilecek bazı yöntemler, asıl sıkıştırma yöntemlerinden önce yapılan ön işlemler olabilir. Hocanın ön işlem örnekleri:

- Seste: kulağın duymadığı frekansları kaldırmak.
- Görüntüde: görüntünün üzerindeki parazitleri kaldırmak.
- Metinde: format bilgilerini ya da çok tekrar eden boşlukları teke indirmek, kayıplı bir sıkıştırmadan söz ediyorsak bütün karakterleri küçük harfe çevirmek.

14. Move-to-Front (MTF) Kodlama

14.1 Fikir

Move-to-front kodlama, herhangi bir yönteme geçmeden önce yapılan bir ön yöntemdir. Lokal adaptif bir kodlama yöntemidir: Bulunduğu bölge içinde hareket eder. Elimizde bir alfabe

vardır. Amaç girdi akışından (input stream) gelen karakterlere kullanım sıklıklarına göre kod atanmasıdır; ama yöntem gereği sıklıklar önceden hesaplanmaz. Bunu move-to-front sağlar.

Kural: İşlenen karakterin alfabedeki sırası (indeks 0'dan başlar) yazılır, sonra karakter alfabenin önüne taşınır ve alfabe kaydırılır.

Hoca örnekteki verinin gerçek veri olmadığını, sonucu anlatabilmek için kullandığı kitaptan alındığını söyledi. Alfabe 8 karakterlidir; herkesin alfabesi 26 ya da 29 karakter olmak zorunda değildir.

14.2 Kodlama örneği

Girdi ve başlangıç alfabesi (ekrandan, 02:20:00):

```
Input stream: a b c d d c b a m n o p p o n m
Alfabe:       {abcdmnop}
```

Adımlar (her satırda karakter işlenmeden önceki alfabe ve yazılan kod):

Girdi	Alfabe (önce)	Kod
a	abcdmnop	0
b	abcdmnop	1
c	bacdmnop	2
d	cbadmnop	3
d	dcbamnop	0
c	dcbamnop	1
b	cdbamnop	2
a	bcdamnop	3
m	abcdmnop	4
n	mabcdnop	5
o	nmabcdop	6
p	onmabcdp	7
p	ponmabcd	0
o	ponmabcd	1
n	opnmabcd	2
m	nopmabcd	3

Alfabenin son hâli `mnopabcd`'dir. Hocanın anlatımı: `a` zaten en öndedir, 0 yazılır, bir şey değişmez. `b` birinci sıradadır, 1 yazılır, `b` öne alınır: `bacdmnop`. `c` ikinci sıradadır, 2 yazılır, öne alınır. İkinci `d` sıfırıncı sıradadır, 0 yazılır. `a`'ya gelindiğinde alfabe `bcdamnop`'tur, `a` üçüncü sıradadır. `m` dördüncü, `n` beşinci, `o` altıncı, `p` yedinci sıradadır; ardından `p` 0, `o` 1, `n` 2, `m` 3 alır.

Çıkan kod dizisi (ekrandan, 02:22:00; python ile doğrulandı):

```
Code: 0 1 2 3 0 1 2 3 4 5 6 7 0 1 2 3
```

Karşı tarafa bu sayı dizisi, herhangi bir yöntemle (hoca "X yöntemi, Y yöntemi" dedi, çünkü yöntemler henüz görülmedi) kodlanarak gönderilir. Hoca sıkıştırırken `0 1 2 3` ve `4 5 6 7`'nin birer desen olarak alınabileceğini, böylece yalnız iki desenin kodlanacağını da söyledi.

14.3 Entropi ve artıklık

Kodlar tek tek sembol olarak alınır, ikişerli ya da dörderli bloklanmaz. 0, 1, 2 ve 3 üçer kez ($3 \cdot 4 = 12$), 4, 5, 6 ve 7 birer kez geçer; toplam 16 kod. Entropi:

$$H = -\left[4 \cdot \tfrac{3}{16} \log_2 \tfrac{3}{16} + 4 \cdot \tfrac{1}{16} \log_2 \tfrac{1}{16}\right] \approx 2{,}81 \text{ bit/karakter}$$

(ekrandan, 02:22:00; python ile 2,811 bulundu.) Slayta göre output stream'deki her kod ortalama 2,81 bitle kodlanır. Kodlar 0 ile 7 arasındadır; 7 en fazla 3 bitle yazılabildiği için sabit kodla her kod 3 bittir ($S = 3$). Slayttaki kod çözme kısmında da `0-7 → 111` diye not düşülmüştür (ekrandan, 02:24:00). Slayta göre code içindeki değerleri kodlamak için Huffman veya aritmetik kodlama kullanılır.

Ders sonrası not: Hoca 02:22:06'da artıklığı "3 eksi 2,81, 0,81" diye söyledi; doğrusu $3 - 2{,}81 = 0{,}19$ bittir.

Hoca bu kodlar için RLE'nin uygun olmadığını da söyledi: RLE uygulanabilir ama sıkıştırma yapmaz, genişletme yapabilir. Daha uygun bir yöntem seçmek gerekir.

14.4 Kod çözme

Slayta göre (ekrandan, 02:24:00) kod çözme işleminde input tersten okunur ve alfabenin ilk hâli oluşturulur. Karşı tarafta iki şey vardır: açılmış kod dizisi ve alfabenin en son hâli (`mnopabcd`). Hoca çözmeyi slayttaki tabloyla gösterdi: Alfabenin son hâlinden başlanır, kodlar sırayla okunur, okunan indeksteki sembol yazılır ve öne taşınır.

Kod	Alfabe (önce)	Çıkan sembol	Alfabe (sonra)
0	mnopabcd	m	mnopabcd
1	mnopabcd	n	nmopabcd
2	nmopabcd	o	onmpabcd
3	onmpabcd	p	ponmabcd
0	ponmabcd	p	ponmabcd
1	ponmabcd	o	opnmabcd
2	opnmabcd	n	nopmabcd
3	nopmabcd	m	mnopabcd
4	mnopabcd	a	amnopbcd
5	amnopbcd	b	bamnopcd
6	bamnopcd	c	cbamnopd
7	cbamnopd	d	dcbamnop
0	dcbamnop	d	dcbamnop
1	dcbamnop	c	cdbamnop
2	cdbamnop	b	bcdamnop
3	bcdamnop	a	abcdmnop

Çıkan semboller `m n o p p o n m a b c d d c b a`'dır; bu girdinin tersidir. En sonda alfabe yeniden başlangıç hâli `abcdmnop`'a döner. Hoca girdinin tersten oluşmasını şöyle açıkladı: Alfabenin en son geldiği hâlden başlandığı için açma ister istemez tersten başlar; bu tamamen simetrik bir algoritmadır, yapılan işlemin tam tersi yapılır.

Ders sonrası not: Slayttaki son satırda alfabe `abcdamnop` yazıyor; hoca fazladan `a`'yı sildi, doğrusu `abcdmnop`'tur. Python ile denendi: Başlangıç alfabesi `abcdmnop` ile kodlar baştan okunursa girdi doğrudan, düz sırayla elde edilir. Slayttaki "son alfabeden başla" yolu bu örnekte doğru sonuç veriyor, çünkü girdinin tersi `mnopabcd` alfabesiyle MTF kodlandığında da aynı kod dizisi (`0 1 2 3 0 1 2 3 4 5 6 7 0 1 2 3`) çıkıyor. Standart MTF çözmede büyük olasılıkla iki taraf başlangıç alfabesini bilir ve kodlar baştan okunur.

14.5 MTF yapılmadan

Hoca aynı girdi ve alfabe ile move-to-front yapmadan kodlamayı gösterdi: Alfabe hiç değişmez, yalnız karakterin alfabedeki yeri yazılır.

```
Girdi: a b c d d c b a m n o p p o n m
Kod:   0 1 2 3 3 2 1 0 4 5 6 7 7 6 5 4
```

(python ile doğrulandı.) Alfabe değişmediği için aynı alfabe karşıya gönderilir ve açıcı her kodun karşılığını doğrudan okur. Bu dizide 0'dan 7'ye her değer iki kez geçer; entropi $-8 \cdot \frac{2}{16} \log_2 \frac{2}{16} = 3$ bittir. MTF'li kodun entropisi 2,81 idi. Demek ki move-to-front entropiyi küçültür ve daha iyi bir sıkıştırma sağlar; yapılmazsa 3 bit gerekir.

15. Kelimelerle Move-to-Front

15.1 Kodlama

MTF yönteminde illa karakter kullanmak gerekmez; aynı yöntem kelimelere de uygulanabilir (ekrandan, 02:32:30). Örnek metin:

```
the boy on my right is the right boy
```

Tabloda iki liste sütunu vardır: "before adding" ve "after adding". Liste başta boştur. Kural: Yeni gelen kelime listenin sonuna eklenir, indeksiyle birlikte çıktıya yazılır ve öne taşınır. Listede zaten bulunan kelime eklenmez; yalnız indeksi yazılır ve öne taşınır.

Kelime	Liste (eklemeden önce)	Liste (ekledikten sonra)	Çıktı
the	()	(the)	0the
boy	(the)	(the, boy)	1boy
on	(boy, the)	(boy, the, on)	2on
my	(on, boy, the)	(on, boy, the, my)	3my
right	(my, on, boy, the)	(my, on, boy, the, right)	4right
is	(right, my, on, boy, the)	(right, my, on, boy, the, is)	5is
the	(is, right, my, on, boy, the)	(is, right, my, on, boy, the)	5
right	(the, is, right, my, on, boy)	(the, is, right, my, on, boy)	2
boy	(right, the, is, my, on, boy)	(right, the, is, my, on, boy)	5

Son liste `(boy, right, the, is, my, on)`'dur (python ile doğrulandı). Kelimeler ilk göründüklerinde pek sıkıştırma elde edilmez. Ama doküman içinde kelimeler bir kez göründükten sonra tekrar ediyorsa yalnız indeks bilgileri yazılır ve bu sıkıştırmaya yol açar.

> **Ders sonrası not:** Hoca 02:32:21'de son `boy` için "alfabenin sonuna getiriyorum" dedi; kodlamada kelime öne taşınır. Slayttaki son liste `(boy, right, the, is, my, on)` bunu gösterir.

15.2 Kod çözme

Açmada (ekrandan, 02:30:00; elle yazılmış tablo) liste yine boş başlar. Çıktıda kelime varsa kelime listeye eklenir, sonra verilen indeksteki kelime çıkarılır ve öne taşınır. Kelime yoksa yalnız indeks vardır; kelime listede zaten bulunuyordur, o indeksteki kelime alınır ve öne taşınır.

- `0the`: `the` eklenir, 0. indeksteki `the` çıkarılır.
- `1boy`: `boy` eklenir (`the, boy`), 1. indeksteki `boy` alınır, öne gelir.
- `2on`, `3my`, `4right`, `5is`: aynı şekilde eklenir ve alınır.
- `5`: eklenecek kelime yok; liste `(is, right, my, on, boy, the)`, 5. indekste `the` vardır.
- `2`: liste `(the, is, right, my, on, boy)`, 2. indekste `right` vardır.
- `5`: liste `(right, the, is, my, on, boy)`, 5. indekste `boy` vardır.

Girdi bitince orijinal metin elde edilmiş olur.

Soru: Orijinal metinde kelimelerin arasında boşluklar var; açılan metin bitişik çıkmaz mı? **Cevap:** Hoca boşlukların istenirse onların da kodlanabileceğini söyledi. Ama boşlukları kodlamak gerekmez: Her kelime arasında bir boşluk olduğu bilindiği için açıcı çıktıyı yazarken her kelimeden sonra bir boşluk bırakabilir.

> **Ek bilgi (derste anlatılmadı):** Move-to-front, büyük olasılıkla en çok Burrows-Wheeler dönüşümünden sonraki adım olarak bilinir (örneğin bzip2). Dönüşüm aynı sembolleri yan yana topladığı için MTF çıktısında küçük indeksler, özellikle 0, çok sık görülür; bu da ardından gelen Huffman ya da aritmetik kodlamanın işini kolaylaştırır.

Örnek Sorular

Soru 1 (ekrandan, 00:40:00): Hilesiz bir parada $P(Y) = P(T) = 1/2$'dir. Sahte bir parada $P(Y) = 1/8$, $P(T) = 7/8$'dir. Her iki durumda yazı ve turanın taşıdığı bilgiyi (kod uzunluğunu) bulun.

> **Çözüm (derste verildi):** Hilesiz: $I(Y) = I(T) = \log_2 2 = 1$ bit. Sahte: $I(Y) = \log_2 8 = 3$ bit, $I(T) = \log_2(8/7) \approx 0{,}193$ bit. Pratikte kesirli bit olmadığı için tura da en az 1 bitle kodlanır.

Soru 2 (ekrandan, 00:44:10): Kaynak alfabesi a_1, a_2, a_3, a_4'ten oluşan bir set için $P(a_1) = 0{,}49$, $P(a_2) = 0{,}25$, $P(a_3) = 0{,}25$, $P(a_4) = 0{,}01$'dir. Setin entropisini bulun. Her sembol 2 bitle kodlanırsa artıklık ne olur?

> **Çözüm (derste verildi):** $H = -(0{,}49 \log_2 0{,}49 + 2 \cdot 0{,}25 \log_2 0{,}25 + 0{,}01 \log_2 0{,}01) = 1{,}57$ bit. $S = 2$, artıklık $|2 - 1{,}57| = 0{,}43$.

Soru 3 (ekrandan, 00:50:00 ve 01:00:00): `1 2 3 2 3 4 5 4 5 6 7 8 9 8 9 10` şeklinde bir sayı dizisi verilmiştir. (a) Bu dizi kodlanırsa ortalama bit uzunluğu ne olur? (b) Diziden bir residual sequence oluşturulursa ortalama kaç bit uzunluğunda kodlanabilir? (c) Alıcı bu diziden orijinal veriye dönebilir mi?

> **Çözüm (derste verildi):** (a) 1, 6, 7, 10 için $P = 1/16$; 2, 3, 4, 5, 8, 9 için $P = 1/8$. $H = 4 \cdot \frac{1}{16} \cdot 4 + 6 \cdot \frac{1}{8} \cdot 3 = 3{,}25$ bit. (b) Rezidü dizisi `1 1 1 -1 1 1 1 -1 1 1 1 1 1 -1 1 1`; $P(1) = 13/16$, $P(-1) = 3/16$, $H = 0{,}70$ bit. (c) Evet, alıcı verinin rezidü olduğunu (başlıktan ya da önceden anlaşarak) bilirse $x_n = x_{n-1} + r_n$ ile döner: $x_1 = 1$, $x_2 = 2$, $x_3 = 3$, $x_4 = 2$, ...

Soru 4 (ekrandan, 01:10:00 ve 01:15:40; derste sözlü, 01:05:36): `1 2 1 2 3 3 3 3 1 2 3 3 3 3 1 2 3 3 1 2` şeklinde verilen sayı dizisini ortalama kaç bitle kodlayabiliriz? Aynı sekansta blokları ikişer sembolden oluşacak şekilde alırsak sonuç değişir mi?

> **Çözüm (derste verildi, düzeltmeyle):** RLE bu diziyi 20 sayıdan 26 sayıya çıkarır, uygun değildir. Tek tek: $P(1) = P(2) = 1/4$, $P(3) = 1/2$, $H = 1{,}5$ bit/sembol, 20 sembol için 30 bit; 2 bitlik sabit kodla artıklık 0,5. İkişerli bloklar: yalnız `12` ve `33`, beşer tane; $P(12) = P(33) = 1/2$ (slaytta $1/4$ yazıyor), $H = 1$ bit/blok, 10 blok için 10 bit. `12` → 0, `33` → 1 ile $S = 1$, artıklık 0.

Soru 5 (ekrandan, 02:10:00): Siyah-beyaz bir görüntü için $P(S_w) = 0{,}8$, $P(S_b) = 0{,}2$, $P(w|b) = 0{,}3$, $P(b|w) = 0{,}01$ olarak verilmiş olsun. Hem Markov model ile hem de olasılık modeli ile sistemin entropisini hesaplayınız.

> **Çözüm (derste verildi):** Olasılık modeli: $H = -(0{,}8 \log_2 0{,}8 + 0{,}2 \log_2 0{,}2) = 0{,}722$ bit. Markov model: $H(S_b) = -(0{,}3 \log_2 0{,}3 + 0{,}7 \log_2 0{,}7) = 0{,}881$ bit, $H(S_w) = -(0{,}01 \log_2 0{,}01 + 0{,}99 \log_2 0{,}99) = 0{,}081$ bit, $H = 0{,}2 \cdot 0{,}881 + 0{,}8 \cdot 0{,}081 = 0{,}241$ bit. Markov model daha düşük entropi verir, ama bu iki durumlu örnekte iki model de pratikte piksel başına 1 bit kullanır.

Soru 6 (ekrandan, 02:20:00, 02:22:00 ve 02:24:00): Input stream `a b c d d c b a m n o p p o n m` ve alfabe `{abcdmnop}` için move-to-front kodunu bulun, entropisini hesaplayın ve kodu çözün. MTF yapılmasaydı entropi ne olurdu?

> **Çözüm (derste verildi, düzeltmeyle):** Kod `0 1 2 3 0 1 2 3 4 5 6 7 0 1 2 3`, alfabenin son hâli `mnopabcd`. $H \approx 2{,}81$ bit/karakter; kodlar 3 bitle yazılırsa artıklık $3 - 2{,}81 = 0{,}19$ (hoca 0,81 dedi). Slayttaki çözmede `mnopabcd`'den başlanıp kodlar sırayla okunur ve `m n o p p o n m a b c d d c b a`, yani girdinin tersi, elde edilir; alfabe `abcdmnop`'a döner. MTF yapılmazsa kod `0 1 2 3 3 2 1 0 4 5 6 7 7 6 5 4` olur ve entropi 3 bittir.

Soru 7 (ekrandan, 02:30:00 ve 02:32:30): `the boy on my right is the right boy` metnini kelime tabanlı move-to-front ile kodlayın ve çözün.

> **Çözüm (derste verildi):** Çıktı `0the 1boy 2on 3my 4right 5is 5 2 5`. Son liste (`boy, right, the, is, my, on`). Çözmede kelime varsa listeye eklenir, verilen indeksteki kelime alınıp öne taşınır; son üç kod 5 → `the`, 2 → `right`, 5 → `boy` verir.

Dersin Toparlaması

Bu derste sıkıştırmanın teorik temeli olan bilgi teorisi işlendi. Bir olayın taşıdığı bilginin olasılığıyla ters orantılı olduğu, bunun $I(A) = -\log_2 P(A)$ ile ölçüldüğü ve olay setinin ortalama bilgisinin entropi $H = -\sum P(A_i) \log_2 P(A_i)$ olduğu görüldü. Entropi teorideki ideal ortalama kod uzunluğudur; pratikte bulunan ortalama mesaj uzunluğu S ile farkı artıklıktır ve iyi bir yöntemde sıfıra yakın olmalıdır. Örneklerle verinin içini tanımanın önemi gösterildi: Fark alarak (rezidü dizisi) 3,25 bitlik entropi 0,70 bite, sembolleri ikişerli bloklayarak 1,5 bitlik entropi blok başına 1 bite indi. Alıcının ön işlemi bilmesi için bilginin başlıkta iletilmesi gerektiği ve statik ile dinamik model ayrımı da anlatıldı.

İkinci bölümde veri modelleri ele alındı: fiziksel model, ignorance ve probability biçimleriyle olasılık modeli ve sembollerin olasılıklarını geçmişteki k sembole bağlı hesaplayan Markov model. Siyah-beyaz görüntü örneğinde Markov modelin entropisi (0,241 bit) olasılık modelinin entropisinden (0,722 bit) düşük çıktı. Son olarak istatistiksel yöntemlere giriş yapıldı ve bir ön işlem olan move-to-front kodlama karakter ve kelime örnekleriyle gösterildi; MTF örnekte entropiyi 3 bitten 2,81 bite indirdi. Hoca sonraki derste prefix kodlardan bahsedileceğini, ileride görülecek yöntemlerin hepsinin prefix kod olduğunu ve prefix olmayan bir kodun sıkıştırılabileceğini ama açılamayacağını söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.